UNIVERSITÉ ABDELMALEK ESSAADI

Faculté des Sciences et Techniques de Tanger

Département Génie Informatique

Cycle ingénieur LSI

Application web de gestion des commandes

Architecture MVC1 et standards JEE

Rapport complet de l'atelier

Réalisé par Assil Elghrib

Deuxième année LSI

Encadrant Pr Lotfi Elaachak

25 octobre 2026

Ce rapport présente la conception et la réalisation d'une application de gestion des clients, des produits et des commandes. Il décrit les choix techniques, les écrans, les vérifications effectuées et la procédure de lancement. Un guide de publication sur GitHub et de remise au professeur complète le dossier.

Technologies utilisées

Java 17 · Servlet 4.0 · JSP 2.3 · JSTL · JDBC · MySQL · Maven · Tomcat 9

Sommaire

1 Contexte et objectifs 3
2 Analyse des besoins et règles métier 4
3 Conception du diagramme de classes 5
4 Schéma relationnel et intégrité des données 6
5 Organisation du projet et configuration Maven 7
6 Couche de persistance et transactions JDBC 8
7 Architecture MVC1 et traitement des requêtes 9
8 Présentation des interfaces de gestion 10
9 Composition et consultation d'une commande 11
10 Validation et résultats des tests 12
11 Installation dans Eclipse et déploiement 13
12 Création du dépôt GitHub et partage du lien 14
13 Remise de l'atelier et bilan 15
14 Références et annexe technique 16

Lecture du rapport

Les parties 1 à 8 présentent la conception et le fonctionnement de l'application. Les parties 9 à 11 expliquent les contrôles et l'installation. Les parties 12 et 13 décrivent la publication du code et les éléments à remettre. Les références et l'annexe technique permettent de retrouver les fichiers et les ressources utilisées.

1 Contexte et objectifs

L'atelier JEE du cycle ingénieur LSI demande une application web capable de gérer les commandes de plusieurs clients. Le travail repose sur les standards Servlet, JSP et JavaBean et sur le modèle MVC1. L'environnement demandé comprend Eclipse, Maven, Tomcat et MySQL. [1]

L'application réalisée rassemble les coordonnées des clients, un catalogue de produits et les commandes qui les relient. Chaque espace dispose des opérations CRUD, c'est-à-dire créer, consulter, modifier et supprimer. Le rapport accompagne le code et explique comment reproduire son exécution.

Correspondance avec les étapes de l'atelier

Étape	Travail demandé	Éléments du projet
1	Concevoir quatre classes métier	Client, Produit, Commande et LigneDeCommande
2	Créer le schéma MySQL	sql/01-schema.sql et sql/02-demo.sql
3	Préparer le projet web Maven	pom.xml, module web 4.0 et dépendance Connector/J
4	Séparer entités, DAO et interfaces	Packages ma.fstt.entities, ma.fstt.dao et ma.fstt.service
5	Réaliser les espaces CRUD	Servlets, beans de page et JSP des trois espaces

Périmètre fonctionnel

Le projet couvre les clients, les produits, les commandes et leurs lignes. Le tableau de bord fournit une vue globale et les listes possèdent une recherche locale. Les prix sont affichés en dirhams marocains, avec le code MAD.

La gestion des stocks, la TVA, les paiements et l'authentification ne font pas partie du périmètre retenu. Le montant présenté au tableau de bord correspond à une somme de commandes non annulées ; il ne représente pas des encaissements.

2 Analyse des besoins et règles métier

Informations manipulées

Entité	Informations principales	Rôle
Client	Identifiant, nom, email, téléphone, adresse	Personne ou organisation passant une commande
Produit	Identifiant, référence, désignation, prix	Article du catalogue
Commande	Identifiant, client, date, statut, lignes	Document regroupant les produits commandés
LigneDeCommande	Identifiant, commande, produit, quantité, prix unitaire	Association entre un produit et une commande

Règles de gestion

- Une commande appartient à un seul client. Un client peut ne posséder aucune commande ou en posséder plusieurs.
- Une commande enregistrée contient de 1 à 50 lignes. Chaque produit ne peut apparaître qu'une seule fois dans cette commande.
- La quantité est un entier compris entre 1 et 100000. Le prix catalogue est positif ou nul, limité à 99999999,99 MAD et à deux décimales.
- Les adresses email des clients et les références des produits sont uniques. Le nom, l'email, la référence et la désignation sont obligatoires.
- Le prix unitaire est mémorisé dans la ligne. Une modification du catalogue ne doit pas changer le montant des commandes déjà enregistrées.
- La suppression d'une commande entraîne celle de ses lignes. Un client ou un produit encore utilisé ne peut pas être supprimé.

Statuts de commande

Les valeurs admises sont EN_COURS, VALIDEE, LIVREE et ANNULEE. L'atelier ne fixe pas de circuit de validation ; l'application permet de choisir librement l'un de ces statuts lors de la création ou de la modification.

3 Conception du diagramme de classes

Le modèle se compose des quatre classes imposées par le sujet. Les entités possèdent des attributs privés, un constructeur sans argument et des accesseurs publics, conformément à l'usage des JavaBeans. Les montants sont manipulés avec BigDecimal et les dates avec LocalDate.

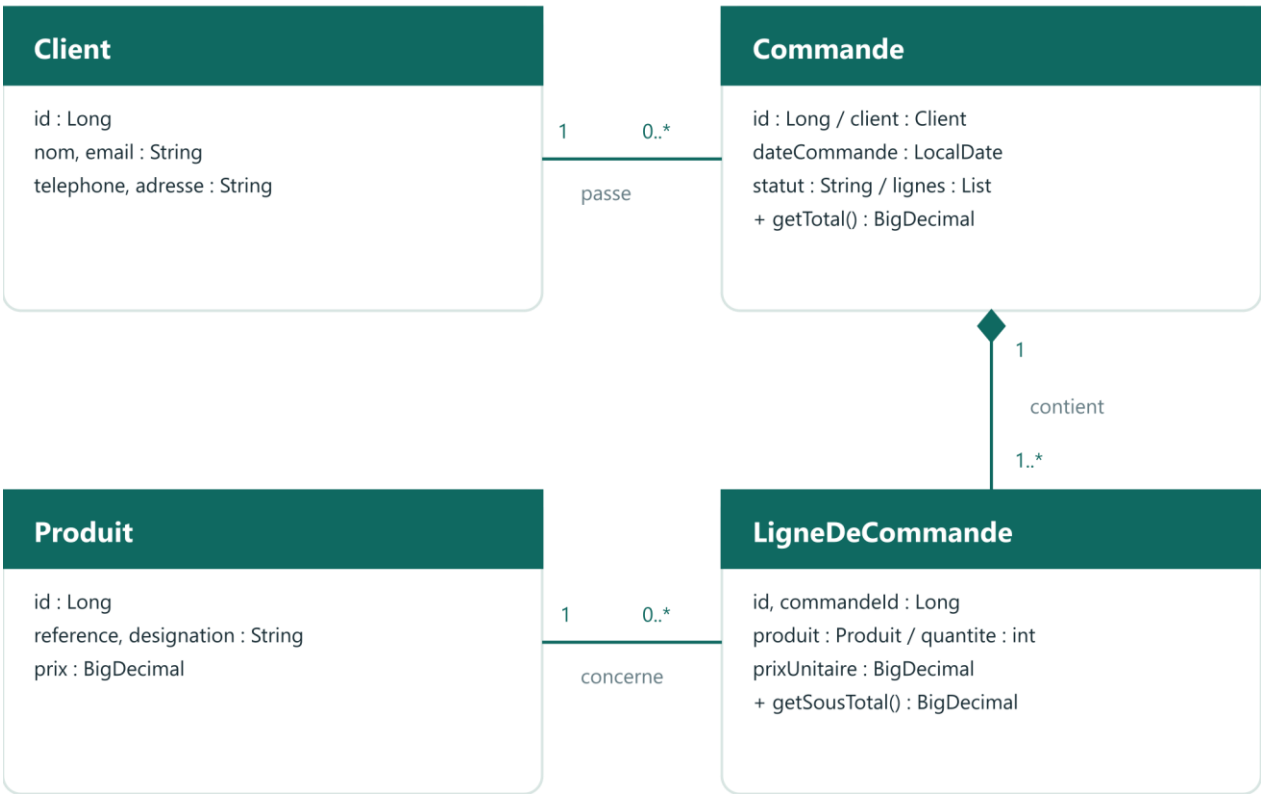


Figure 1 Diagramme de classes et cardinalités

Lecture des associations

La relation Client vers Commande est de type un à plusieurs. La relation Commande vers LigneDeCommande est une composition : une ligne n'existe pas indépendamment de la commande qui la porte. Chaque ligne référence exactement un produit ; un produit peut figurer dans plusieurs commandes.

Calcul des montants

Le sous-total d'une ligne est égal à sa quantité multipliée par son prix unitaire historique. Le total de la commande correspond à la somme de ses sous-totaux. Ces valeurs sont calculées par getSousTotal() et getTotal() plutôt que stockées dans une colonne supplémentaire.

```
Sous-total = quantité × prix unitaire historique
Total commande = somme des sous-totaux
```

La collection de lignes est portée par Commande. Le lien vers le client est porté par la commande et le lien vers le produit par chaque ligne. Ce choix limite les associations bidirectionnelles inutiles dans un atelier JDBC.

4 Schéma relationnel et intégrité des données

Le diagramme est traduit en quatre tables. Les associations deviennent des clés étrangères. Le schéma complet est fourni dans sql/01-schema.sql ; les données fictives facultatives se trouvent dans sql/02-demo.sql.

Table	Colonnes et types	Contraintes
client	id BIGINT ; nom VARCHAR(120) ; email VARCHAR(180) ; telephone VARCHAR(30) ; adresse VARCHAR(255)	PK id ; UNIQUE email
produit	id BIGINT ; reference VARCHAR(40) ; designation VARCHAR(180) ; prix DECIMAL(12,2)	PK id ; UNIQUE reference ; contrôle du prix
commande	id BIGINT ; client_id BIGINT ; date_commande DATE ; statut VARCHAR(20)	PK id ; FK client_id ; contrôle du statut
ligne_commande	id BIGINT ; commande_id BIGINT ; produit_id BIGINT ; quantite INT ; prix_unitaire DECIMAL(12,2)	PK id ; deux FK ; UNIQUE(commande_id, produit_id) ; contrôles quantité et prix

Choix des clés et des contraintes

Les clés primaires utilisent AUTO_INCREMENT. Toutes les colonnes sont NOT NULL ; le téléphone et l'adresse peuvent contenir une chaîne vide. Les contraintes d'unicité préviennent les doublons d'email, de référence et de produit dans une même commande.

Les relations vers le client et le produit utilisent ON DELETE RESTRICT. La relation des lignes vers la commande utilise ON DELETE CASCADE. Les index sur commande.client_id et ligne_commande.produit_id accompagnent les recherches par relation.

Limite des contraintes SQL

Une clé étrangère garantit qu'une référence existe, mais elle ne garantit pas qu'une commande possède au moins une ligne. Cette règle, comme la limite de 50 lignes, est vérifiée dans CommandeDAO avant l'enregistrement. MySQL 8.0.16 ou supérieur et le moteur InnoDB sont retenus pour les contrôles et les transactions.

5 Organisation du projet et configuration Maven

Structure des sources

```
src/main/java/ma/fstt/
  entities/   Client, Produit, Commande, LigneDeCommande
  service/    Contrats de persistance
  dao/        Implémentations JDBC
  config/     Database
  web/        Servlets, beans de page, filtre et utilitaires
src/main/webapp/
  index.jsp
  WEB-INF/pages/   clients.jsp, produits.jsp, commandes.jsp
  WEB-INF/jspf/    En-tête et pied de page
  WEB-INF/web.xml
  assets/         Feuille de style et JavaScript
src/test/java/ma/fstt/   Tests DAO et HTTP
```

Dépendances principales

Composant	Version du projet	Usage
Java	Cible 17	Compilation des sources
Servlet API	4.0.1, provided	API fournie par Tomcat
JSTL	1.2	Affichage avec les balises JSP
MySQL Connector/J	8.4.0	Connexion JDBC à MySQL
JUnit Jupiter et H2	5.12.1 et 2.3.232	Tests uniquement
Tomcat embarqué	9.0.123	Tests HTTP et compilation des JSP

Le packaging Maven est war et le nom final est gestion-commandes.war. Le module déclaré dans web.xml est en version 4.0. Les plugins compiler, surefire et war assurent respectivement la compilation, les tests et la création du livrable.

Connector/J est déclaré avec les coordonnées com.mysql:mysql-connector-j. La documentation MySQL confirme ces identifiants Maven. Tomcat 9 correspond aux API Servlet 4.0 et JSP 2.3 utilisées ici. [4][5]

6 Couche de persistance et transactions JDBC

Contrats des DAO

Les interfaces du package ma.fstt.service définissent les opérations de persistance. ClientService, ProduitService et CommandeService étendent le contrat générique CrudService. Les DAO fournissent les implémentations JDBC ; les beans de page les utilisent sans écrire de SQL.

```
public interface CrudService<T> {
    List<T> findAll() throws SQLException;
    Optional<T> findById(long id) throws SQLException;
    void save(T entity) throws SQLException;
    void delete(long id) throws SQLException;
}
```

La méthode save insère un objet sans identifiant ou met à jour un objet existant. Les clés générées sont récupérées après insertion. Les requêtes préparées séparent le texte SQL des paramètres. Les blocs try-with-resources ferment les connexions, les instructions et les résultats.

Enregistrement atomique d'une commande

CommandeDAO ouvre une connexion et désactive l'autocommit. Pour une modification, une lecture SELECT FOR UPDATE verrouille l'en-tête. Les prix historiques sont récupérés, puis chaque produit est vérifié. L'en-tête est inséré ou modifié et LigneDeCommandeDAO remplace les lignes avec la même connexion.

```
// Enchaînement simplifié de CommandeDAO.save
c.setAutoCommit(false);
try {
    // Vérifier les produits et enregistrer l'en-tête
    lignesDAO.replace(c, e.getId(), e.getLignes());
    c.commit();
} catch (SQLException | RuntimeException ex) {
    c.rollback();
    throw ex;
}
```

Un échec annule l'ensemble des écritures. La méthode replace ne valide pas sa propre transaction : elle participe à celle de la commande. Cette organisation évite un en-tête enregistré avec des lignes incomplètes.

7 Architecture MVC1 et traitement des requêtes

Dans le modèle JSP Model 1, la JSP prend en charge le traitement d'une requête et sa réponse, éventuellement en s'appuyant sur des JavaBeans. Dans Model 2, un servlet contrôleur organise le traitement et choisit la vue. [3]

Le projet conserve le traitement dans les JSP et leurs beans de page. Les trois servlets servent d'alias d'URL et transfèrent toujours vers la JSP de leur espace. Cette adaptation répond à la présence simultanée de MVC1, Servlets et JSP dans le sujet ; elle ne met pas en place un contrôleur métier central.

```
Navigateur
  -> SecurityFilter
  -> Servlet alias /clients, /produits ou /commandes
  -> JSP de l'espace
  -> Bean XxxPage
  -> Interface de service et DAO
  -> Base de données
```

Rôle du servlet et de la JSP

Le servlet de l'espace client transfère la requête à la page privée. Le bean de page réalise la validation et l'appel au DAO lorsque la JSP le déclenche.

```
// Extrait de ClientServlet
request.getRequestDispatcher(
    "/WEB-INF/pages/clients.jsp"
).forward(request, response);

// Début de clients.jsp
<%@ page import="ma.fstt.web.ClientPage" %>
<% if (new ClientPage().handle(request,response)) return; %>
```

Les expressions EL et les balises JSTL parcourent les objets et affichent les données. Les valeurs saisies sont échappées avec c:out. Après une écriture réussie, une redirection vers une requête GET évite que le rafraîchissement ne répète l'opération.

Les DAO sont instanciés explicitement. Les annotations @Inject et @ApplicationScoped n'étant pas utilisées, aucun moteur CDI supplémentaire n'est nécessaire dans cette version.

8 Présentation des interfaces de gestion

L'interface comprend une navigation latérale, un tableau de bord et trois espaces. Les écrans s'adaptent à la largeur disponible. Les listes de clients, de produits et de commandes proposent une recherche qui filtre les lignes déjà chargées dans le navigateur.

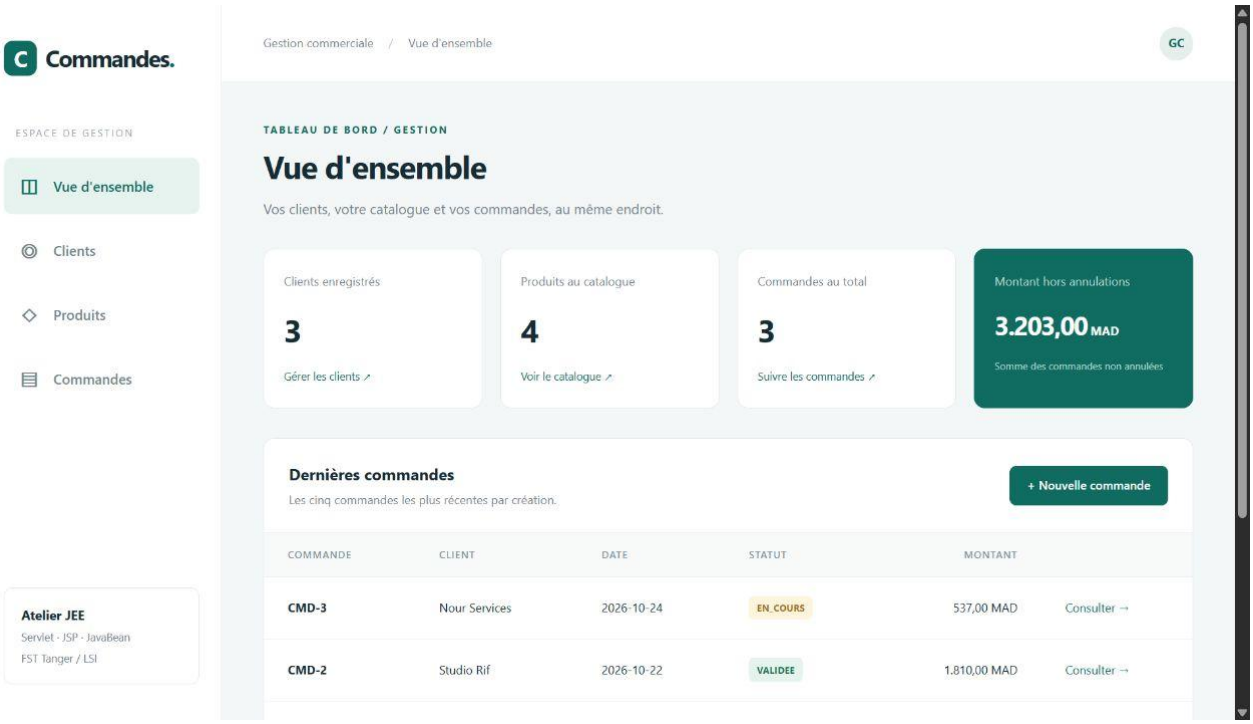


Figure 2 Tableau de bord avec les données fictives initiales

Espace	Opérations disponibles
Clients	Ajouter et modifier les coordonnées ; consulter la liste ; supprimer un client non référencé
Produits	Ajouter et modifier une référence, une désignation et un prix ; consulter et supprimer un produit libre
Commandes	Créer une commande ; consulter son détail ; changer le client, la date, le statut ou les lignes ; supprimer la commande

Avec les données initiales, le tableau de bord affiche 3 clients, 4 produits et 3 commandes. Le montant hors annulations est de 3203,00 MAD, soit 856,00 + 1810,00 + 537,00 MAD.

9 Composition et consultation d’une commande

Le formulaire permet de choisir un client, une date et un statut, puis d’ajouter ou retirer des lignes. Le JavaScript affiche une estimation des montants pendant la saisie. Le serveur recalcule le résultat à partir de prix fiables avant l’enregistrement.

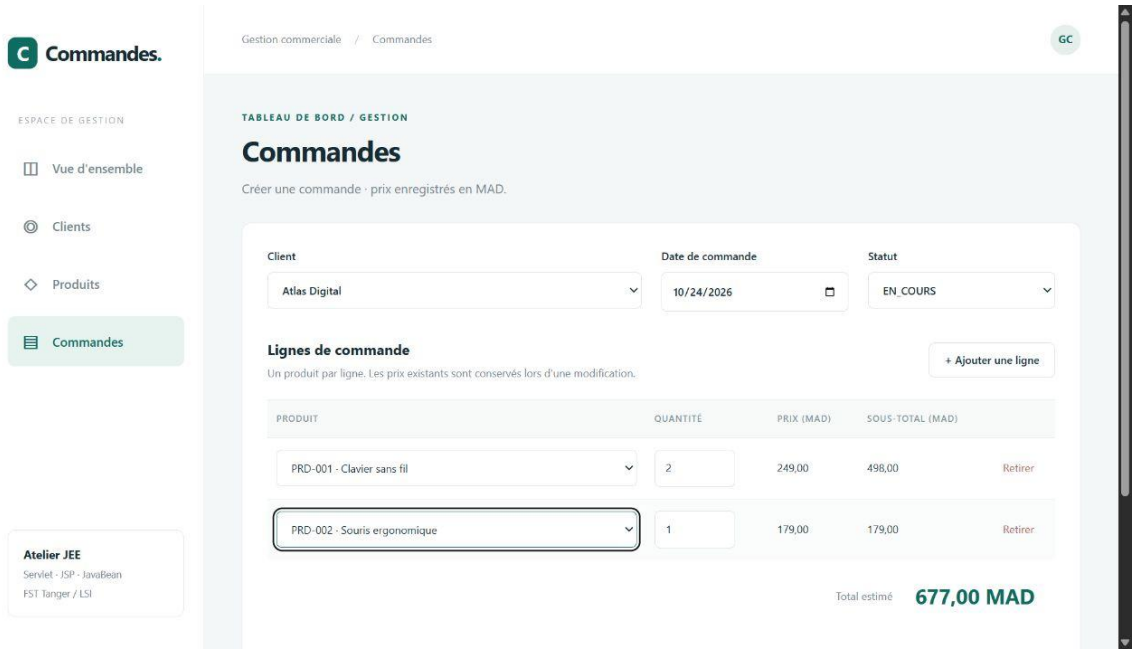


Figure 3 Formulaire avec deux produits et calcul de 677,00 MAD

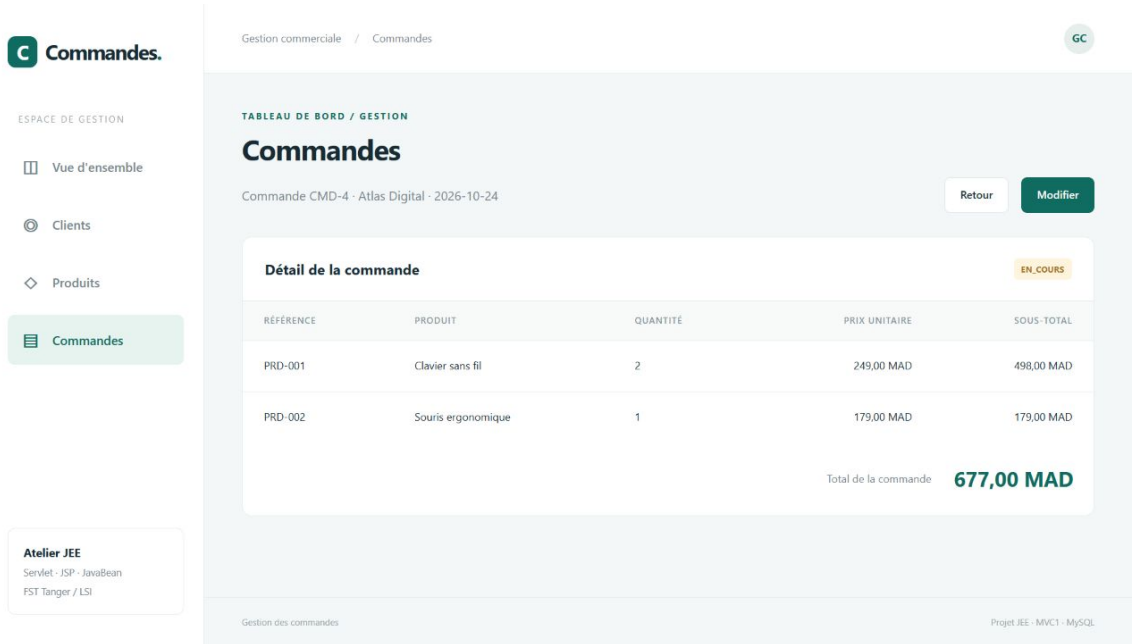


Figure 4 Détail de la commande enregistrée et total confirmé

Le scénario vérifié comprend 2 claviers à 249,00 MAD et 1 souris à 179,00 MAD : 2 × 249,00 + 179,00 = 677,00 MAD. La commande de cette capture provient de la base temporaire de vérification ; le script initial conserve ses trois commandes de démonstration.

10 Validation et résultats des tests

Contrôles de saisie et de requête

Les contrôles côté serveur vérifient les champs obligatoires, les longueurs, l'email, les identifiants, la date, les statuts, le prix et les quantités. Les produits d'une commande doivent être distincts. Le prix transmis par le navigateur n'est jamais accepté comme prix de référence.

Les écritures utilisent POST et exigent un jeton CSRF associé à la session. Les requêtes préparées limitent le risque d'injection SQL et l'échappement HTML protège l'affichage des données. Les erreurs de contraintes produisent un message explicite ; les détails techniques sont journalisés sur le serveur.

Résultats de la vérification automatisée

La vérification du 24 octobre 2026 a exécuté Maven verify avec le JDK 25.0.2, tout en ciblant Java 17. Elle se termine par BUILD SUCCESS. Les résultats proviennent du journal de compilation et de docs/VERIFICATION.md. [2]

Suite	Tests	Échecs	Erreurs
DaoTest	12	0	0
WebIntegrationTest	5	0	0
Total	17	0	0

Scénarios couverts

Les tests DAO vérifient les opérations CRUD, les doublons, les clés étrangères, les calculs et la conservation des prix. Un test force une erreur SQL lors du remplacement des lignes, puis contrôle la restauration de l'ancien statut et des anciennes lignes par rollback.

Les tests HTTP utilisent Tomcat 9 embarqué et compilent réellement les JSP. Ils contrôlent les pages, les formulaires, le cycle d'une commande, le rejet d'un jeton CSRF absent, l'échappement HTML et les erreurs attendues. Le contrôle visuel a également confirmé le calcul dynamique et l'enregistrement des deux produits.

Portée des résultats

La base des tests est H2 en mode MySQL. Aucun serveur MySQL réel n'a été utilisé pour cette vérification. Les scripts et la configuration MySQL sont fournis ; une recette locale après déploiement reste nécessaire pour confirmer le fonctionnement dans l'environnement de remise.

11 Installation dans Eclipse et déploiement

Préparer MySQL

Utiliser un JDK 17 ou supérieur, Maven 3.9 ou supérieur, Tomcat 9 et MySQL 8.0.16 ou supérieur. Créer une base vide en utf8mb4, puis importer le schéma. Le second script ajoute les données fictives et ne doit être importé qu'une fois dans une base vide.

```
CREATE DATABASE gestion_commandes
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE gestion_commandes;
SOURCE C:/chemin/gestion-commandes-jee/sql/01-schema.sql;
SOURCE C:/chemin/gestion-commandes-jee/sql/02-demo.sql;
```

SOURCE est une commande du client mysql. Avec MySQL Workbench, ouvrir et exécuter les deux scripts après avoir sélectionné la base. Un compte commandes_app peut être créé avec les droits SELECT, INSERT, UPDATE et DELETE sur cette base.

Configurer la connexion

Dans le processus qui démarre Tomcat, définir DB_URL, DB_USER et DB_PASSWORD. Le fichier db.properties.example permet aussi de créer une configuration locale avant compilation. Ce fichier local contenant le mot de passe ne doit pas être publié.

```
DB_URL=jdbc:mysql://localhost:3306/gestion_commandes
DB_USER=commandes_app
DB_PASSWORD=mot_de_passe_local
```

Importer et lancer

1. Dans Eclipse, choisir File > Import > Maven > Existing Maven Projects, puis sélectionner le dossier contenant pom.xml.
2. Choisir le JDK 17 ou supérieur et ajouter un serveur Apache Tomcat v9.0. Vérifier le facet Dynamic Web Module 4.0.
3. Configurer les variables DB_* dans le lancement du serveur et exécuter Run on Server. En ligne de commande, mvn clean verify produit le WAR.
4. Pour un déploiement manuel, copier gestion-commandes.war dans webapps de Tomcat 9 et démarrer le serveur.

Adresse locale : http://localhost:8080/gestion-commandes/. Le WAR précompilé est disponible dans livraison/gestion-commandes.war. Le projet utilise javax.servlet ; Tomcat 10 ou 11 demanderait une migration vers jakarta.servlet.

12 Création du dépôt GitHub et partage du lien

Le dépôt doit permettre au professeur de consulter le code, les scripts et les instructions. Un dépôt n'a pas encore été publié pour ce projet ; les étapes ci-dessous expliquent comment le créer depuis le navigateur. [6][7]

Préparer les fichiers à publier

Extraire l'archive ZIP, puis ouvrir gestion-commandes-jee. Préparer les dossiers src, sql et docs, ainsi que pom.xml, README.md et .gitignore. Ajouter ce rapport après sa vérification. Garder les mots de passe, db.properties et le dossier target hors de l'envoi ; le navigateur ne filtre pas automatiquement les fichiers selon .gitignore.

Créer le dépôt et envoyer le code

1. Se connecter à github.com, puis ouvrir github.com/new. Choisir son compte comme propriétaire et saisir gestion-commandes-jee comme nom.
2. Choisir Public si les consignes du professeur le permettent. Avec Private, inviter le professeur pour lui donner accès. Laisser les options de fichiers initiaux désactivées, puis cliquer sur Create repository.
3. Sur la page vide, utiliser le lien d'envoi de fichiers existants. Sur un dépôt déjà initialisé, choisir Add file puis Upload files.
4. Glisser les dossiers et fichiers préparés dans la page, en conservant leur structure. Écrire un message tel que Ajout du projet atelier JEE.
5. Valider l'enregistrement avec Commit changes, ou Propose changes si une branche est proposée. Dans ce second cas, terminer la demande de fusion vers la branche principale.
6. Revenir à la page principale et vérifier que les sources Java, les JSP, le SQL et le README sont consultables. Copier son adresse depuis la barre du navigateur.

Le lien à transmettre est l'adresse de la page du dépôt, et non le lien github.com/new, un chemin local ou l'adresse d'un seul fichier. Le lien GitHub permet de consulter le code ; l'application doit toujours être exécutée sur Tomcat avec sa base de données.

Vérification avant partage

Ouvrir le lien dans une autre session pour vérifier l'accès public, ou confirmer l'invitation du professeur si le dépôt est privé. Vérifier également que le rapport PDF et les instructions de lancement sont présents. L'adresse réelle du dépôt pourra ensuite être ajoutée au message de remise.

13 Remise de l’atelier et bilan

Éléments à transmettre au professeur

Élément	Contenu attendu	Mode de remise
Code source	Java, JSP, assets, pom.xml, scripts SQL et README	Lien du dépôt GitHub
Rapport	Auteur, conception, base, architecture, captures et tests	Ce rapport au format PDF
Application compilée	gestion-commandes.war	Facultatif, si demandé ou ajouté à l’archive
Démonstration	Clients, produits, commande de plusieurs lignes et contrôles	Présentation locale si demandée

Contrôles avant l’envoi

- Vérifier le nom Assil Elghrib et la mention deuxième année LSI sur la couverture.
- Lancer l’application avec MySQL et Tomcat 9, puis contrôler au moins un cycle de création, modification et suppression.
- Vérifier les diagrammes, les captures, le rapport PDF et le README dans le dépôt.
- Copier le lien réel du dépôt et vérifier les droits d’accès du professeur.

Exemple de message de remise

Objet : Remise de l’atelier JEE de gestion des commandes

Bonjour Monsieur,

Veuillez trouver ci-joint mon rapport de l’atelier JEE portant sur la gestion des commandes avec MVC1. Le code source, les scripts SQL et les instructions d’installation sont disponibles dans le dépôt GitHub dont je vous transmets le lien.

Cordialement,
Assil Elghrib
Deuxième année LSI

Bilan du projet

Le projet met en pratique les JavaBeans, JDBC et les JSP avec JSTL. Les lignes préservent les prix historiques et les transactions assurent la cohérence des commandes. Les 17 tests vérifient les comportements principaux. La recette MySQL et la publication GitHub restent à réaliser avant la remise. Un usage public demanderait une authentification, des autorisations et une gestion adaptée des connexions.

14 Références et annexe technique

Sources du rapport

[1] Sujet fourni par Pr Lotfi Elaachak, Application web basée sur MVC et Standards JEE, FST Tanger, cycle ingénieur LSI. Les cinq étapes du sujet définissent le périmètre de l'atelier.

[2] Sources du projet gestion-commandes-jee, notamment README.md, pom.xml, sql/01-schema.sql, docs/VERIFICATION.md et les suites de tests. Résultats de vérification datés du 24 octobre 2026.

[3] Oracle — Servlets and JSP Pages Best Practices

[4] Apache Tomcat 9 — Documentation

[5] MySQL — Installing Connector/J Using Maven

[6] GitHub — Creating a new repository

[7] GitHub — Adding a file to a repository

Ressources en ligne consultées le 25 octobre 2026. Les légendes des captures décrivent des données fictives de démonstration.

Fichiers à consulter pour approfondir

Sujet technique	Fichier du projet
Connexion JDBC	src/main/java/ma/fstt/config/Database.java
Enregistrement transactionnel	src/main/java/ma/fstt/dao/CommandeDAO.java
Traitement du formulaire	src/main/java/ma/fstt/web/CommandePage.java
Formulaire et détail	src/main/webapp/WEB-INF/pages/commandes.jsp
Contraintes de la base	sql/01-schema.sql
Contrôles HTTP	src/test/java/ma/fstt/WebIntegrationTest.java